Teams How-To Manual

This guide is a practical, end-to-end manual for using the Teams capability in `ragent`.

It covers:

- Team concepts and data layout
- Blueprints — pre-built team templates
- TUI command workflows
- Tool-driven workflows
- Work context delegation
- Example operating patterns
- Configuration and file examples
- Troubleshooting and safe cleanup

---

1) What Teams are for

Teams let one lead session coordinate multiple teammate agents with:

- Shared task queue (`tasks.json`)
- Per-member mailbox messaging
- Persistent team configuration (`config.json`)
- Blueprint-based team templates with auto-spawning
- Work context propagation — teammates know exactly what code to target

Use Teams when you want parallel execution (for example: API/UI/tests split, or security/perf/test review swarms) while keeping one lead in control.

---

2) Team architecture and storage

Teams are persisted on disk:

- Project-local (preferred): `[PROJECT]/.ragent/teams/<team-name>/`
- Global fallback: `~/.ragent/teams/<team-name>/`

Typical structure:

```
.ragent/
  teams/
    feature-squad/
      config.json
      tasks.json
      README.md
      mailbox/
        tm-001.ndjson
        tm-002.ndjson
```

Key files:

- `config.json`: team metadata, members, settings, lifecycle state
- `tasks.json`: shared task list and status

- `README.md`: team documentation (copied from blueprint if used)
- `mailbox/*.ndjson`: teammate/lead messages

---

3) Blueprints

Blueprints are pre-built team templates that define which teammates to spawn and what seed tasks to create. When you create a team with a blueprint, all teammates are spawned automatically with their role-specific prompts.

Blueprint storage locations

Blueprints are searched in priority order (closest wins):

1. **Project-local:** `[PROJECT]/.ragent/blueprints/teams/<name>/`
2. **User-config (XDG):** `~/.config/ragent/blueprints/teams/<name>/`
3. **User-global (legacy):** `~/.ragent/blueprints/teams/<name>/`

Blueprint directory structure

```
.ragent/blueprints/teams/code-review/
  README.md              # Team description (copied to team directory)
  spawn-prompts.json     # Teammate definitions with role-specific prompts
  task-seed.json         # Initial tasks seeded on team creation
```

spawn-prompts.json format

Defines teammates to auto-spawn when the team is created:

```
[
  {
    "tool_name": "team_spawn",
    "teammate_name": "security-reviewer",
    "agent_type": "general",
    "prompt": "Perform a focused security review of the codebase..."
  },
  {
    "tool_name": "team_spawn",
    "teammate_name": "test-reviewer",
    "agent_type": "general",
    "prompt": "Review test coverage and identify gaps..."
  }
]
```

task-seed.json format

Defines initial tasks created when the team is set up:

```
[
  {
    "tool": "team_task_create",
```

```
    "input": {
      "title": "Audit authentication boundaries",
      "description": "Confirm session/auth checks protect privileged actions."
    }
  }
]
```

Both `"input"` and `"args"` keys are supported for tool arguments.

Browsing installed blueprints

Use the `/team blueprint` slash command:

```
/team blueprint              # List all installed blueprints
/team blueprint code-review  # Show detailed summary of a specific blueprint
```

The list view shows a table with blueprint name, scope (project/global), teammate count, task count, and description. The detail view shows the full README, teammate table (name, type, prompt), seed tasks table (title, description), and usage instructions.

---

4) Quick start in the TUI

Create a team from a blueprint (recommended)

```
/team create code-review
```

This creates a team using the `code-review` blueprint, auto-spawns all defined teammates with their prompts, and seeds the initial task list. The blueprint parameter is required.

When the LLM calls `team_create`, it should **always pass a `context` parameter** with the user's specific request — which files/directories to review, what to produce, and where to write output. This context is prepended to every teammate's spawn prompt so they know exactly what to work on.

Wait for teammates to finish

After creation, the lead should call `team_wait` to block until all teammates complete their initial work. Do **not** use `wait_agents` for teammates — that only tracks `new_agent` sub-agents.

Inspect team state

```
/team                # Same as /team status
/team status         # Show active team with member table
/team tasks          # Show shared task list
/team show           # List all registered teams
/team show myteam    # Show details for a specific team
```

View teammate output

In the Teams panel, click the `[T]` button next to a teammate to open their output view. This shows all tool calls, results, and messages in the same rich format as the primary Messages panel — including inline diffs, result summaries, and reasoning blocks.

Message teammates

```
/team message api-builder Please prioritize auth error handling.
```

End-of-work actions

```
/team clear                 # clear shared tasks list only
/team close                 # close active team in this session only
/team delete feature-squad  # delete persisted team from disk
/team cleanup               # cleanup active team (conservative guardrails)
```

---

5) Complete slash command reference

The full `/team` syntax and subcommand table (`help`, `status`, `show`, `create`, `close`, `delete`, `blueprint`, `message`, `tasks`, `clear`, `cleanup`, `forcecleanup`, `focus`) is in slashcommands/team.md.

Alias: `/teams ...` routes to `/team ...` (for example `/teams help`, `/teams blueprint`).

---

6) Tool-based workflow (lead + teammates)

Teams are fully operable via tools. TUI commands are convenience wrappers.

6.1 Create team with context

```
{
  "tool": "team_create",
  "input": {
    "blueprint": "code-review",
    "context": "Review the crates/ragent-server directory for security issues, test coverage gaps
    "project_local": true
  }
}
```

The `context` parameter is critical — it tells every auto-spawned teammate **exactly what code** to target. Without it, teammates only receive their generic role prompt from the blueprint.

If the team already exists, `team_create` gracefully recovers and re-applies the blueprint.

The TUI's `/team create` creates the team itself and then seeds the blueprint with `seed_only: true`, which applies the blueprint to the existing team and never creates (or re-creates) its directory. That keeps a seeding call from resurrecting a team directory that a concurrent `/team clear` or `/team forcecleanup` has just removed.

6.2 Wait for teammates

After `team_create` spawns all teammates, call `team_wait` immediately:

```
{
  "tool": "team_wait",
  "input": {
```

```
    "team_name": "code-review-20260329-13-10-25",
    "timeout_secs": 300
  }
}
```

This blocks until all teammates become idle. **Do NOT use `wait_agents`** — that only tracks `new_agent` sub-agents, not team members.

6.3 Spawn additional teammates (manual)

```
{
  "tool": "team_spawn",
  "input": {
    "team_name": "feature-squad",
    "teammate_name": "api-builder",
    "agent_type": "general",
    "prompt": "Implement backend endpoint + validation for the feature."
  }
}
```

Note: When using blueprints, teammates are spawned automatically — do NOT re-spawn them manually.

6.4 Create tasks

```
{
  "tool": "team_task_create",
  "input": {
    "team_name": "feature-squad",
    "title": "Implement API endpoints",
    "description": "Create API routes and data access for feature flow.",
    "depends_on": []
  }
}
```

6.5 Claim and complete tasks (teammate sessions)

- Claim next available task: `team_task_claim`
- Complete claimed task: `team_task_complete` — accepted only from the task's assigned agent or the lead. A refusal does not hand the task over; it reports `rejection_reason` (`not_owner`, `not_claimed`, `already_completed`) plus the current owner, and leaves the task list untouched.
- Lead withdraws work: `team_task_cancel` (lead-only). Only `pending` and `in_progress` tasks are cancellable; the owner is notified through its mailbox; the task stays in the list with status `cancelled` rather than being deleted, so the team records that the work was dropped rather than done.

6.6 Check status and communicate

- `team_status` — Full team report: member states, task summary, each member's last five status transitions (from `status-history.jsonl`), `Undelivered messages` stranded in a terminated teammate's mailbox, `Plans awaiting a verdict`, and `Hook failures` for quality gates that could not run

- `team_task_list` — List all tasks with status
- `team_message` — Send direct message to a teammate
- `team_read_messages` — Read inbox messages
- `team_broadcast` — Send message to all teammates
- `team_assign_task` — Lead assigns a specific task to a teammate
- `team_idle` — Teammate signals it has no more work
- `team_cleanup` — Lead tears the team down after all teammates have stopped. Refused while any member is live or shutting down; parked (`idle`, `plan_pending`, `blocked`, `suspended`) and terminal members never block. `force: true` is the only override, and `/team cleanup` and `/team delete` share the same guard.

6.6.1 State machines the runtime enforces

Three rules are enforced rather than merely documented, and each refusal is visible in the tool result and the event stream:

- **One member-status writer.** `TeamManager::set_member_status` validates the move against the transition table, applies the destination status's side effects, persists the roster once, appends one record to `status-history.jsonl`, and publishes exactly one `TeammateStatusChanged`. `MemberStatus::is_terminal` / `is_parked` / `is_live` are the predicates every guard uses, so time a parked member spends waiting is never evidence of a wedge.
- **One task-status transition table.** `TaskStatus::can_transition_to` and `transition_task` are the single writer of `Task::status`, so the claim and completion paths cannot disagree about which moves are possible. Every `tasks.json` write runs `TaskList::validate()` first and refuses a duplicate id, a dangling `depends_on` edge, or a dependency cycle, naming the offending task and chain.
- **Hook failures are recorded.** A quality-gate hook that cannot run (invalid command, spawn failure, 30-second timeout) appends a record to `hook-failures.jsonl` and its count surfaces in `team_status`, so an inert hook is never mistaken for one that ran and approved.

6.6.2 Who is calling, and where the runtime lives

Every team tool except `team_create` resolves its caller first (`tool::team_caller`). It uses the session's team context when that names the team; otherwise it falls back to the team's `config.json`, where the caller is the lead if the session is `lead_session_id` and a teammate if it is a member's `session_id`. A session that is neither is refused.

Lead-only tools refuse a teammate: `team_spawn`, `team_task_create`, `team_assign_task`, `team_approve_plan`, `team_shutdown_teammate`, `team_cleanup` and `team_wait`. Before this check landed a teammate could approve its own plan or force-delete the whole team, and a session outside the team could read another team's mailbox. `team_wait` without `team_name` defaults to the caller's own team.

The team runtime is bound before tool dispatch, in every entry point. Only the TUI ever created a `TeamManager`, so in `ragent run`, `ragent serve` and ACP mode `team_spawn` reported a teammate as "queued" but never started it. The session processor now binds the runtime for the team its session leads (`SessionProcessor::ensure_team_manager`), reconciles members queued in `Spawning`, and starts the watchdog. A `team_spawn` that runs before the runtime exists (the same step as `team_create`) returns an error telling you to call it again, instead of a false "queued".

The runtime is rebindable, not first-team-forever. The processor holds it in a `TeamManagerSlot`:

- a manager whose team is gone is unbound atomically (`clear_if`), so a manager another session bound meanwhile is kept;
- the processor keeps a live manager of another team rather than replacing it, so concurrent `serve` sessions leading different teams do not flip the binding on every step. The TUI, which has a single user, rebinds when its active team changes; teammates already running keep the old manager alive until they finish;
- `spawn_teammate` refuses a `team_name` its manager does not drive;
- `team_shutdown_teammate` and `team_task_cancel` use the bound manager only when it drives the named team, and otherwise act on disk.

Roster writes are atomic. Every read-modify-write of `config.json` goes through `TeamStore::update`, which holds the exclusive lock from read to write and skips the write when nothing changed. Two teammates changing status at once can no longer read the same snapshot and silently discard one change.

6.7 Plan approval (optional)

When `settings.require_plan_approval` is `true` in the team config, teammates must get sign-off before mutating work:

1. Teammate calls `team_submit_plan` with its plan.
2. The lead reviews and calls `team_approve_plan` (accept or reject). Member state tracks `plan_status` / `plan_request_id` for correlation.

6.8 Team memory

Teammates can persist shared notes via `team_memory_read` / `team_memory_write`, backed by the SQLite memory store (default bucket `MEMORY.md`). Access is gated by the member's `memory` scope (`"user"` or `"project"` in the member profile); members with `"none"` cannot use team memory.

6.9 Per-teammate model override

Team members accept a `model_override` field (and `team_spawn` a matching parameter) to pin a teammate to a specific `provider:model` instead of the globally selected model.

6.10 Team hooks

`TeamSettings.hooks` accepts hook entries (`HookEvent` + command) that the runtime executes via `run_team_hook` at team lifecycle events — the same `HookEntry` format as global session hooks.

---

7) Work context delegation

When a team is created and assigned work, the specific details about what code needs to be addressed must flow through to every teammate. This is handled by the `context` parameter on `team_create`.

How context flows

1. User asks: *"Review the crates/ragent-server directory for security issues"*
2. Lead calls `team_create` with `context: "Review the crates/ragent-server directory..."`
3. Each teammate's spawn prompt from the blueprint gets the context prepended:

```
## Work Context
Review the crates/ragent-server directory for security issues, test coverage,
and performance problems. Write findings to COMPLIANCE.md.

## Your Role
Perform a focused security review of the assigned code...
```

4. Teammates immediately know which files to read and what to produce.

Best practices for context

- Include **specific directories/files** to target
- Specify **output format** (e.g., "write to COMPLIANCE.md")
- Mention **what to look for** (security, performance, tests)
- Keep it concise but complete — this is the teammates' primary instruction

---

8) Team panel (TUI)

The Teams panel displays the active team in a table format:

```
Team: code-review-20260329-13-10-25  (lead + 3 teammates)
  id         name                              status      elapsed  steps  claimed  done
  b729b590   general                           active      6m41s    14     -        -
  tm-001     security-reviewer                 idle        6m27s    22     0        0    [T]
  tm-002     test-reviewer                     idle        6m27s    25     0        0    [T]
  tm-003     performance-reviewer              idle        6m27s    7      0        0    [T]
```

Columns:

- **id**: Short agent ID (last 8 chars of session UUID for lead, tm-NNN for teammates)
- **name**: Teammate name (35 chars max) or agent type for lead
- **status**: `active`, `working`, `idle`, `failed`, `spawning` (a `stopped` state also exists on disk for shutdown teammates and is filtered from the panel)
- **elapsed**: Time since teammate was created
- **steps**: Number of tool calls executed. This count reflects individual tool invocations, not agent loop iterations. A single loop step may contain zero or multiple tool calls (for example, when parallel tool calls are enabled), so the displayed number may differ from the loop-step counter used internally for log tagging. Since v1.0.73, the TUI step log also shows these tracked sub-agent/teammate tool calls with an `[agent-tag]` prefix.
- **claimed**: Number of tasks claimed from the shared task list
- **done**: Number of tasks completed
- **[T]**: Click to open teammate's output view

Teammate output view

Clicking `[T]` opens a scrollable overlay showing the teammate's full execution history — all tool calls, read results, write operations, and assistant messages. The output uses the same rich format as the primary Messages panel:

- Colored status dots (green = success, red = error)

- Inline diff stats for edit/patch operations
- Line counts for read/write operations
- File tree rendering for list operations

The output view updates incrementally during execution — you can watch a teammate's progress in real-time without waiting for completion.

---

9) Example operation playbooks

9.1 Blueprint-based code review (recommended)

1. Browse available blueprints:

```
/team blueprint
```

2. Inspect the code-review blueprint:

```
/team blueprint code-review
```

3. Ask ragent to create and run the team:

```
Create a code-review team and have it review the crates/ragent-server directory
for security issues, test gaps, and performance problems. Write the output to
COMPLIANCE.md in the crates/ragent-server folder.
```

The LLM will: - Call `team_create` with `blueprint="code-review"` and `context="Review the crates/ragent-server directory..."` - All 3 teammates auto-spawn with the context prepended to their prompts - Call `team_wait` to block until all teammates finish - Call `team_status` to collect findings - Aggregate results into the requested output file

9.2 Parallel feature delivery (API/UI/Tests)

1. Lead creates team:

```
/team create parallel-feature
```

2. Lead spawns 3 teammates:

- `api-builder`
- `ui-builder`
- `test-owner`

3. Lead seeds tasks with dependency chain:

- API implementation
- UI implementation (depends on API)
- Regression tests (depends on API + UI)

4. Track in TUI:

```
/team status
/team tasks
```

5. Team completes work, lead finalizes:

```
/team clear
/team close
```

---

10) Configuration and file examples

This section shows representative file formats used by Teams.

10.1 Example team `config.json`

```json
{
  "schema_version": 1,
  "name": "feature-squad",
  "lead_session_id": "sess-lead-123",
  "created_at": "2026-03-19T10:00:00Z",
  "updated_at": "2026-03-19T10:21:00Z",
  "status": "active",
  "members": [
    {
      "name": "api-builder",
      "agent_id": "tm-001",
      "session_id": "sess-tm-001",
      "agent_type": "general",
      "status": "working",
      "current_task_id": "task-001",
      "plan_status": "none",
      "model_override": null,
      "memory_scope": "project",
      "created_at": "2026-03-19T10:01:00Z"
    }
  ],
  "settings": {
    "max_teammates": 8,
    "require_plan_approval": false,
    "auto_claim_tasks": true,
    "hooks": []
  }
}
```

> **Schema note:** `TeamConfig` and `Task` are deserialised with `deny_unknown_fields`, so hand-edited files containing unknown keys (e.g. a typo'd field name) are rejected on load. Fields omitted above all carry `#[serde(default)]` values.

10.2 Example `tasks.json`

```json
{
  "team_name": "feature-squad",
  "tasks": [
    {
```

```
      "id": "task-001",
      "title": "Implement API endpoints",
      "description": "Add backend endpoint + data wiring.",
      "status": "completed",
      "assigned_to": "tm-001",
      "depends_on": [],
      "created_at": "2026-03-19T10:02:00Z",
      "claimed_at": "2026-03-19T10:03:00Z",
      "completed_at": "2026-03-19T10:20:00Z"
    },
    {
      "id": "task-002",
      "title": "Implement UI flow",
      "description": "Build screens and interactions.",
      "status": "in_progress",
      "assigned_to": "tm-002",
      "depends_on": ["task-001"],
      "created_at": "2026-03-19T10:04:00Z",
      "claimed_at": "2026-03-19T10:21:00Z",
      "completed_at": null
    }
  ]
}
```

10.3 Example blueprint spawn-prompts.json

```
[
  {
    "tool_name": "team_spawn",
    "teammate_name": "security-reviewer",
    "agent_type": "general",
    "prompt": "Perform a focused security review. Check for auth gaps, injection vectors, unsafe
  },
  {
    "tool_name": "team_spawn",
    "teammate_name": "test-reviewer",
    "agent_type": "general",
    "prompt": "Review test coverage. Identify untested paths, missing edge cases, and fragile ass
  },
  {
    "tool_name": "team_spawn",
    "teammate_name": "performance-reviewer",
    "agent_type": "general",
    "prompt": "Analyze performance and resource usage. Focus on expensive loops, repeated I/O, un
  }
]
```

10.4 Example task-seed.json

```
[
  {
    "tool": "team_task_create",
    "input": {
      "title": "Audit authentication boundaries",
      "description": "Confirm session/auth checks protect privileged actions."
    }
  },
  {
    "tool": "team_task_create",
    "input": {
      "title": "Audit command execution safety",
      "description": "Review shell/tool invocation paths for injection risks."
    }
  }
]
```

---

11) Reading `/team tasks` output

The command shows a table like:

```
ID           Title                              Status        Assignee
-----------  ---------------------------------  ------------  --------
task-001     Implement API endpoints            completed     tm-001
task-002     Implement UI flow                  in-progress   tm-002
task-003     Add feature regression tests       pending       ---
```

Status meanings (canonical on-disk values in `tasks.json` are snake_case; the table renders them with hyphens):

- `pending`: waiting to be claimed
- `in_progress`: currently being worked (displayed as `in-progress`)
- `completed`: finished
- `cancelled`: intentionally stopped

Completion is idempotent for the completing agent: the same agent calling `team_task_complete` again succeeds without mutation (the task records `completed_by`), but a *different* agent completing an already-completed task is rejected.

---

12) Operational guidance and best practices

- **Always use blueprints** for repeatable team patterns — they save time and reduce errors.
- **Always pass `context`** when creating teams — teammates can't read your mind about which files to target.
- **Call `team_wait`** immediately after `team_create` — don't try to read results before teammates finish.

- **Don't re-spawn blueprint teammates** — they are spawned automatically by `team_create`.
- Keep teammate prompts role-specific and narrow.
- Seed tasks before heavy teammate execution.
- Use `depends_on` to control sequencing across streams.
- Ask teammates to send short, frequent progress messages.
- Use `/team clear` when you want to reset task planning without deleting the team.
- Use `/team close` to exit team context safely in your current session.
- Use `/team delete <name>` only when you are done and want full removal.

---

13) Troubleshooting

"No active team"

Cause: no current team selected in this session.

Fix:

```
/team show          # list available teams
/team create <blueprint>   # create new team
```

"Failed to open team"

Cause: team name not found in project-local/global team directories.

Fix:

- Verify team name spelling
- Confirm the team exists under `.ragent/teams/` or `~/.ragent/teams/`

"teammate(s) still active — shut them down first"

Cause: destructive operation requested while teammates still in `working` state.

Fix:

- Request teammate shutdown (`team_shutdown_teammate` + `team_shutdown_ack`)
- Retry cleanup/delete after teammates are idle/stopped
- Alternatively, use `/team forcecleanup` to forcibly deactivate and remove all teammates (prompts for confirmation before proceeding)

"teammate(s) still active" with `/team forcecleanup`

Cause: `/team cleanup` failed because teammates are still working, and you need to force them offline.

Fix:

```
/team forcecleanup
```

This will prompt for confirmation, then deactivate all active teammates and clear the team state. Each active teammate is shut down through the unified immediate-shutdown path (agent loops and poll joins aborted, the member marked `Stopped` on disk) before the team directory is removed, so no background write can recreate the directory mid-teardown. Use with caution — any in-progress work will be lost.

"parse tasks.json: EOF while parsing"

Cause: `tasks.json` exists but is empty (0 bytes). This can happen if a concurrent operation created the file but didn't write content.

Fix: The system now handles empty `tasks.json` files gracefully. If you encounter this with an older version, delete the empty file and retry:

```
/team clear
```

Teammates resolving wrong paths

Cause: In older versions, teammates could resolve relative paths against the team directory instead of the project root.

Fix: Ensure you are running the latest version. Teammate sessions now inherit the lead session's working directory (the project root).

"0 lines read" in teammate output

Cause: In older versions, tool metadata (line counts, file counts) was not persisted to the message store, so the teammate output view couldn't display result summaries.

Fix: Ensure you are running the latest version. Tool metadata is now merged into the persisted message, and the output view displays correct line counts, diff stats, and summaries.

---

14) Safety and lifecycle recommendations

- Prefer `close` before `delete`.
- Use `clear` to reset work queue while preserving team composition/history.
- Keep cleanup conservative unless you are sure no teammate is running.
- Avoid manual edits in team files while sessions are active.

---

15) Swarm decomposition (`/swarm`)

The `/swarm` command provides fleet-style auto-decomposition. It analyses your prompt, breaks it into independent subtasks with dependency edges, creates an ephemeral team, and orchestrates parallel execution.

Swarm commands

`/swarm <prompt>`, `/swarm --agent <type> <prompt>`, `/swarm status`, `/swarm cancel` and `/swarm help`; see slashcommands/swarm.md.

How it works

1. The swarm analyses your prompt using the selected LLM model.
2. It breaks the goal into independent subtasks with dependency edges.
3. It creates an ephemeral team and spawns teammates for each subtask.

4. Teammates work in parallel, claiming tasks as they become available (respecting dependency ordering).
5. The lead monitors progress via `/swarm status` and can cancel with `/swarm cancel`.

Example

```
/swarm Review the crates/ragent-server directory for security vulnerabilities, test coverage gaps
```

With a default agent type:

```
/swarm --agent code-review Review the authentication module for OWASP Top 10 vulnerabilities
```

Swarm vs. manual teams

Feature	`/team create`	`/swarm`
Task decomposition	Manual (blueprint seed tasks)	Automatic (LLM decomposes prompt)
Team creation	Requires a blueprint	Ephemeral, auto-created
Teammate spawning	Blueprint-driven or manual	Automatic per subtask
Dependency management	Manual in task-seed.json	Automatic from decomposition
Best for	Structured, repeatable reviews	Ad-hoc, goal-driven parallel work

---

16) Related docs and examples

- Quickstart: `QUICKSTART.md` (Teams section)
- Blueprint location: `[PROJECT]/.ragent/blueprints/teams/`, `~/.config/ragent/blueprints/teams/`, or `~/.ragent/blueprints/teams/`
- Blueprints installed in this repository:
  - `.ragent/blueprints/teams/code-review/`
  - `.ragent/blueprints/teams/parallel-feature/`
  - `.ragent/blueprints/teams/debate/`

Each blueprint directory holds a `README.md`, a spawn-prompt document (`spawn-prompts.json` - the roster `team_create` executes; `parallel-feature` carries the prose `spawn-prompts.md` variant) and a `task-seed.json` (the shared task list `team_create` seeds). Copy one into your own project's `.ragent/blueprints/teams/` to use it; a hook script can be attached by pointing the blueprint's hook entry at any executable (e.g. a `teammate-idle.sh` gate).